Instrucțiuni de lucru în virgulă mobilă

Unitatea de virgulă mobilă

- *Floating Point Unit* - FPU
- inițial - separată de procesor
  - inclusă în procesor începând cu Intel 486
- lucrează intern cu operanzi în precizie extinsă (80 de biți)
- orice operand de tip `float` sau `double` este convertit la această reprezentare

Regiștrii interni (1)

- 8 regiștri pe 80 de biți
  - denumiri: ST(0) ... ST(7)
- sunt organizați într-o stivă
  - ST(0) este întotdeauna vârful stivei
  - referit și ca ST
- orice operație de scriere în stivă schimbă ordinea (și numele) regiștrilor
  - fostul ST(0) devine ST(1) etc.

Regiștrii interni (2)

- la orice instrucțiune, ST(0) este unul dintre operanzi
  - sau singurul, la operațiile unare
- celălalt operand poate fi, după caz
  - o locație de memorie (valoare întreagă sau în virgulă mobilă)
  - un alt registru ST(i)
  - niciodată o constantă

Regiștrii interni (3)

- introducerea în stivă - prin instrucțiuni de încărcare a unor valori
  - locații de memorie
  - unele constante predefinite
- scoaterea din stivă
  - multe instrucțiuni de procesare au și o variantă care elimină ST(0) din stivă
  - numele acestor instrucțiuni se termină cu "p"

Încărcare din memorie

- sintaxa

`fld` *`adresa`*

`fild` *`adresa`*

- introduce în stivă operandul, care este
  - în primul caz - o variabilă în virgulă mobilă (`float`, `double`)
  - în al doilea caz - o variabilă întreagă (`short int`, `int`)

Exemple

```
float f;
int i;
_asm {
  fld f
  fild i
  fld dword ptr [ebp+8]
}
```

Încărcare constantă predefinită

- sintaxa

```
fldz
fld1
fldpi
```

- introduc în stivă, respectiv, valorile 0, 1, π
- există și alte constante (logaritmice)

Scriere în memorie

- sintaxa

`fst` *`adresa`*

`fstp` *`adresa`*

  – scriere ST(0) într-o variabilă în virgulă mobilă

`fist` *`adresa`*

`fistp` *`adresa`*

  – scriere ST(0) într-o variabilă întreagă

Exemple

```
float f;
int i;
_asm fstp f;
```

- copiază ST(0) în `f` și îl elimină din stivă
- fostul ST(1) devine ST(0) etc.

```
_asm fist i;
```

- copiază ST(0) în `i`, păstrându-l pe stivă

Interschimbare regiștri

- sintaxa

```
fxch st(i)
```

- `i` este o valoare constantă între 1 și 7
- schimbă între ele valorile regiștrilor ST(0) și ST(i)

Adunare (1)

- unul dintre operanzi este întotdeauna ST(0)
  - de obicei și destinația rezultatului
- celălalt poate fi
  - locație de memorie (virgulă mobilă sau întreg)
  - alt registru din stiva FPU
- în ultimul caz, rezultatul poate fi scris în celălalt registru, în loc de ST(0)
  - eventual cu eliminarea ST(0) din stivă

Adunare (2)

- sintaxa

`fadd` *`adresa`* //virgulă mobilă

`fiadd` *`adresa`* //întreg

`fadd st(0),st(i)` //ST(0)=ST(0)+ST(i)

`fadd st(i),st(0)` //ST(i)=ST(i)+ST(0)

`faddp st(i),st(0)` //cu scoatere ST(0) din stivă

Scădere (1)

- similar cu adunarea
- ST(0) este de obicei descăzutul și rezultatul
- celălalt operand
  - locație de memorie (virgulă mobilă sau întreg)
  - alt registru din stiva FPU
- în ultimul caz, celălalt registru poate fi descăzutul și destinația rezultatului
  - eventual cu eliminarea ST(0) din stivă

Scădere (2)

- sintaxa

`fsub` *`adresa`* //virgulă mobilă

`fisub` *`adresa`* //întreg

`fsub st(0),st(i)` //ST(0)=ST(0)-ST(i)

`fsub st(i),st(0)` //ST(i)=ST(i)-ST(0)

`fsubp st(i),st(0)` //cu scoatere ST(0) din stivă

Scădere inversă (1)

- ordinea operanzilor este invers față de operația de scădere obișnuită
  - se scade primul operand din al doilea
  - dar rezultatul este scris tot în primul operand

Scădere inversă (2)

- sintaxa

`fsubr` *`adresa`* //virgulă mobilă

`fisubr` *`adresa`* //întreg

`fsubr st(0),st(i)` //ST(0)=ST(i)-ST(0)

`fsubr st(i),st(0)` //ST(i)=ST(0)-ST(i)

`fsubrp st(i),st(0)` //scoate ST(0) din stivă

Înmulțire

- similar cu adunarea

`fmul` *`adresa`* //virgulă mobilă

`fimul` *`adresa`* //întreg

`fmul st(0),st(i)` //ST(0)=ST(0)*ST(i)

`fmul st(i),st(0)` //ST(i)=ST(i)*ST(0)

`fmulp st(i),st(0)` //cu scoatere ST(0) din stivă

Împărțire (1)

- împărțire în virgulă mobilă
- ST(0) - de obicei deîmpărțitul și rezultatul
- celălalt operand
  - locație de memorie (virgulă mobilă sau întreg)
  - alt registru din stiva FPU
- în ultimul caz, celălalt registru poate fi deîmpărțitul și destinația rezultatului
  - eventual cu eliminarea ST(0) din stivă

Împărțire (2)

- sintaxa

`fdiv` *`adresa`* //virgulă mobilă

`fidiv` *`adresa`* //întreg

`fdiv st(0),st(i)` //ST(0)=ST(0)/ST(i)

`fdiv st(i),st(0)` //ST(i)=ST(i)/ST(0)

`fdivp st(i),st(0)` //cu scoatere ST(0) din stivă

Împărțire inversă (1)

- similar cu scăderea inversă
- al doilea operand este deîmpărțitul, iar primul este împărțitorul
- dar rezultatul este scris tot în primul operand

Împărțire inversă (2)

- sintaxa

`fdivr` *`adresa`* //virgulă mobilă

`fidivr` *`adresa`* //întreg

`fdivr st(0),st(i)` //ST(0)=ST(i)/ST(0)

`fdivr st(i),st(0)` //ST(i)=ST(0)/ST(i)

`fdivrp st(i),st(0)` //scoate ST(0) din stivă

Diverse operații unare

- ST(0) este singurul operand (implicit)
  - și destinația rezultatului

`fabs`

  - calculează valoarea absolută a ST(0)

`fchs`

  - schimbă semnul valorii din ST(0)

`fsqrt`

  - caculează rădăcina pătrată din ST(0)

Comparații

- compară ST(0) cu alt registru din stiva FPU
  - ST(0) este primul termen al comparației
- setează indicatorii de condiții
- sintaxa

```
fcomi st(0),st(i)
fcomip st(0),st(i)
```

  - în a doua variantă, ST(0) este scos din stivă

Valorile indicatorilor de condiții

relație	Zero	Carry
ST(0) > ST(i)	0	0
ST(0) < ST(i)	0	1
ST(0) = ST(i)	1	0